

SecureRx — Documentación criptográfica

Recetas electrónicas con autenticidad, confidencialidad y no repudio. Este documento describe el material criptográfico del sistema y señala el archivo exacto donde vive cada pieza.

La pila se apoya en cuatro familias: **cifrado autenticado simétrico** (AES-128-GCM) para el contenido de la receta, **envoltura asimétrica de clave** (RSA-OAEP-SHA256) para entregar la clave a destinatarios, **firma digital** (ECDSA P-256 con SHA3-256) para autenticidad y no repudio de actos clínicos, y una **CA interna X.509 v3** que ata las llaves públicas a una identidad. Encima se apilan **Argon2id** para passwords, **JWT HS256** para sesión y **TLS 1.3** en el transporte.

Frontend, nginx y aspectos no criptográficos del sistema sólo se mencionan tangencialmente. El foco aquí son los primitivos y dónde se invocan en el código.

1 · AES-128-GCM — cifrado autenticado del contenido de la receta

Cada receta se cifra una sola vez con una **clave de datos efímera** (DEK, 128 bits) y un IV aleatorio de 96 bits. GCM produce ciphertext más un **TAG** de 128 bits que protege ambos: ciphertext y AAD. Cualquier alteración posterior (un bit cambiado, un swap del AAD, rebinding a otra receta) hace que la verificación del TAG falle y el descifrado lance excepción.

Decisiones de diseño

■ **DEK efímera por receta:** cero reutilización de claves entre documentos, así un compromiso aislado no propaga.

■ **IV nunca se repite por DEK:** como la DEK también es única por receta, la condición (DEK, IV) único es trivial. Aun así generamos IV fresco aleatorio con `os.urandom(12)`.

■ **AAD denso:** `id_receta`, `id_doctor`, `id_paciente`, `fecha_creacion`, `dispensaciones_permitidas`, `version`. Esto evita rebinding del criptograma a otra fila.

■ **TAG de 128 bits:** el máximo que GCM soporta — sin truncar.

■ **Primitivo** `services/crypto/aes_gcm.py`

■ **Uso en flujo** `services/receta_cifrado.py` · `routers/recetas_crear.py`

```
dek = os.urandom(16)
iv = os.urandom(12)
ct, tag = aes_gcm_encrypt(dek, iv, plaintext, aad)
# persistir: ciphertext, tag, iv, aad — y desechar la DEK
```

2 · RSA-OAEP-SHA256 — envoltura de la DEK por destinatario

La DEK no se guarda nunca en claro: se **envuelve** con la pública RSA-2048 del paciente y de cada farmacia autorizada. Cada envoltura produce un blob (~256 bytes) que **solo el dueño de la priv RSA puede desenvelopar**. Para una receta dirigida a paciente + N farmacias hay 1 + N envolturas almacenadas: `c_wrap_pac` en la fila de la receta y `c_wrap_far` en `receta_acceso_farmacias`.

Por qué OAEP

PKCS#1 v1.5 es vulnerable a ataques de oráculo de relleno (Bleichenbacher). OAEP con SHA-256 elimina esa clase de ataque al exigir un padding verificable y aleatorizado por operación. Además, **el mismo plaintext (DEK) envuelto dos veces produce blobs distintos** — el atacante no puede emparejar envolturas por simple inspección.

■ **Primitivo** `services/crypto/rsa_oaep.py`

■ **Uso en flujo** `services/receta_cifrado.py` · `services/receta_descifrado.py`

```
c_wrap = rsa_oaep_encrypt(pub_rsa_pem, dek)
# más adelante, el destinatario:
dek = rsa_oaep_decrypt(priv_rsa_pem, c_wrap)
# luego AES-GCM descifra el ciphertext
```

3 · ECDSA P-256 + SHA3-256 — firmas de autenticidad y no repudio

Las firmas digitales en SecureRx **nunca son sobre el texto crudo** — se firman *JSON canónicos deterministas* (R, sello, M_cancel, manifiesto de acuse). Esto garantiza que la verificación es estable frente a re-serIALIZACIÓN, reordenamiento de claves o espacios en blanco. El algoritmo es **ECDSA sobre la curva NIST P-256**, con SHA3-256 como hash de entrada — Keccak, ortogonal a SHA-2.

Cuatro firmas distintas a lo largo del ciclo de vida

- 1) **S_D** — firma del médico sobre el documento R canónico al emitir. Es el ancla del documento.
- 2) **S_F** — sello del farmacéutico sobre el evento de dispensación (id_evento, num_dispensación, timestamp).
- 3) **Acuse** — firma del paciente sobre el manifiesto del evento. Es la confirmación no-repudiable de que recibió el medicamento.
- 4) **S_cancel** — firma del médico sobre M_cancel cuando una receta se anula. Persiste como prueba criptográfica de la cancelación.

■ Primitivo `services/crypto/signatures.py`

■ JSON canónico `services/canonical.py`

■ Documentos firmables `services/canonical_receta.py`

■ Verificación `routers/recetas_verificar.py`

```
r_bytes = build_R(...) # JSON canónico bytes
firma = ecdsa_sign(ec_priv_pem, r_bytes) # SHA3-256 internamente
ok = ecdsa_verify(ec_pub_pem, r_bytes, firma)
```

4 · CA interna X.509 v3 — quien firma identidad a las llaves públicas

Toda llave pública del sistema queda atestiguada por un **certificado X.509 v3** firmado por la CA interna. Esto convierte una pública anónima en una identidad verificable (Dr. X, Farmacia Y, paciente Z) y permite a cualquiera validar firmas y envolturas sabiendo a quién pertenecen las llaves.

Topología de la CA

■ **CA raíz self-signed**, EC P-256, vigencia 10 años. Su privada vive en `ca/ca_key.pem` con permisos 0600 y *nunca* sale del servidor.

■ **BasicConstraints(ca=True, path_length=0)** en la raíz: solo puede firmar end-entity, jamás sub-CAs. Modelo de un nivel — sin delegación.

■ Cada usuario aprobado recibe **dos certificados**: uno EC con `KeyUsage = digitalSignature + nonRepudiation` para firmar, y uno RSA con `KeyUsage = keyEncipherment` para envoltura. **KeyUsage es crítica** — un cliente que no entienda la extensión debe rechazar el cert.

■ **BasicConstraints(ca=False)** en cada cert de usuario: si roban la priv de un médico, no puede emitir certs para terceros.

■ CA + emisión `services/crypto/ca.py`

■ Endpoint público raíz `routers/health.py` (GET `/health/ca/certificate`)

■ Emisión administrada `routers/admin.py` (aprobar solicitud)

5 · Argon2id — hashing de contraseñas

Las passwords **nunca** se guardan ni se transmiten en claro. Tampoco se hashean con SHA-256 ni bcrypt: usamos **Argon2id**, ganador de la *Password Hashing Competition* y resistente a ataques GPU/ASIC por su consumo intensivo de memoria. El salt se incluye dentro del propio string Argon2 (formato PHC) — no hay campo separado en BD.

Parámetros

■ **m_cost** = 64 MiB de memoria por intento

■ **t_cost** = 3 iteraciones

■ **parallelism** = 1

■ Adicionalmente, la BD guarda salt_pw (32 bytes) por exigencia explícita del spec — pero Argon2 ya tiene su salt embebido en el hash.

■ **Primitivo** `services/crypto/argon2_pw.py`

■ **Uso** `routers/usuarios.py (registro · login · reset)`

6 · JWT HS256 — sesión

Tras autenticarse, el backend emite un JWT **HS256** firmado con JWT_SECRET (256 bits, en variable de entorno). El payload lleva sub (id), rol y exp. Cada request lo trae en Authorization: Bearer <token> y el dependency auth_required lo verifica antes de tocar la BD.

Por qué HS256 y no RS256

El backend es monolítico: el mismo proceso emite y verifica los JWTs, no hay servicios externos que necesiten validar sin acceso al secret. HS256 es más rápido y la rotación del secret es trivial. Si se agregaran microservicios independientes, migrar a RS256 sería natural.

■ **Primitivo** `services/crypto/jwt_service.py`

■ **Dependency** `auth.py (auth_required, require_roles)`

7 · Flujo criptográfico end-to-end de una receta

§4 Emisión (médico firma)

1. El médico arma R canónico con todo el contenido sensible.
2. Genera **DEK aleatoria (128 bits)** y **IV (96 bits)**.
3. Construye AAD canónico (id_receta, id_doctor, fecha, ...).
4. **Cifra**: AES-128-GCM.encrypt(DEK, IV, R, AAD) → (ciphertext, TAG).
5. **Firma**: S_D = ECDSA(ec_priv_médico, R).
6. **Envuelve la DEK** con la pública RSA del paciente y de cada farmacia autorizada: c_wrap_X = RSA-OAEP(pub_RSA_X, DEK).
7. **Persiste** ciphertext, TAG, IV, AAD, c_wrap_pac, c_wrap_far[N], S_D, y el hash SHA3-256 de R como huella pública.
8. **Borra la DEK** de memoria.

■ **Endpoint** `routers/recetas_crear.py`

§5 Dispensación (farma sella)

1. El farma aporta su priv RSA (en header durante la sesión).
2. **Desenvuelve la DEK**: DEK = RSA-OAEP-dec(priv_RSA_farma, c_wrap_far).
3. **Descifra el ciphertext** con AES-GCM y verifica TAG + AAD.
4. **Verifica S_D** con la pub EC del médico (vía cert).
5. Construye **sello canónico** (id_evento, núm_dispensación, timestamp).
6. **Firma S_F**: S_F = ECDSA(ec_priv_farma, sello).
7. Persiste el evento + S_F. La DEK se desecha.

■ **Endpoint** `routers/recetas_dispensar.py`

§6 Acuse del paciente

1. El paciente desenvuelve la DEK con su priv RSA.
2. Verifica la firma S_F del farma.
3. **Firma el manifiesto del evento** con su priv EC: `acuse = ECDSA(ec_priv_paciente, M_evento)`.
4. Se persiste como prueba no-repudiable de que recibió el medicamento.

■ **Endpoint** `routers/recetas_dispensar.py` (**firmar-paciente**)

§8 Cancelación · §9 Nueva versión

Cancelar: el médico firma M_cancel con su priv EC; la receta no se borra, queda en estado *cancelada* con la firma S_cancel como prueba. **Nueva versión:** se crea una receta nueva (DEK, IV, AAD propios) con `parent_id = id_anterior`; la original pasa a estado *sustituída*. Cero edición in-place — todo el historial queda criptográficamente atado.

■ **Cancelación** `routers/recetas_cancelar.py`

■ **Nueva versión** `routers/recetas_nueva_version.py`

8 · Capas no criptográficas (mención breve)

■ **nginx** termina TLS 1.3 exclusivo (sin fallback a 1.2), aplica HSTS, CSP, X-Frame-Options. El cert TLS es self-signed EC P-256 para local; en producción se reemplazaría por uno de una CA pública. Esta capa **no firma ni cifra payloads** — solo el transporte.

■ **Frontend** (React) custodia las priv del usuario en memoria de sesión, las envía en un header dedicado X-Priv-Keys solo cuando el backend necesita desenvolver. Nunca persiste las priv en localStorage ni cookies.

■ **TLS config** `eprescriptions-backend/nginx.conf + gen-cert.sh`

■ **Custodia priv en cliente** `frontend/src/components/ui/SessionKeyPicker.jsx`

9 · Mapa rápido — cripto-relevantes

Para localizar cualquier pieza criptográfica en el código:

- **AES-128-GCM** `services/crypto/aes_gcm.py`
- **RSA-OAEP-SHA256** `services/crypto/rsa_oaep.py`
- **ECDSA P-256 + SHA3-256** `services/crypto/signatures.py`
- **Generación/parseo de llaves** `services/crypto/keys.py`
- **Argon2id (password hashing)** `services/crypto/argon2_pw.py`
- **JWT HS256 (sesión)** `services/crypto/jwt_service.py`
- **CA interna X.509 v3** `services/crypto/ca.py`
- **JSON canónico determinista** `services/crypto/canonical.py`
- **Documentos firmables (R, AAD)** `services/canonical_receta.py`
- **Composición cifrado+envoltura** `services/receta_cifrado.py`
- **Descifrado simétrico+RSA** `services/receta_descifrado.py`
- **Hidratador → frontend** `services/hidratador.py`
- **Emisión / firma de receta** `routers/recetas_crear.py`
- **Dispensación / sello** `routers/recetas_dispensar.py`
- **Cancelación firmada** `routers/recetas_cancelar.py`
- **Nueva versión (sustitución)** `routers/recetas_nueva_version.py`
- **Verificación criptográfica** `routers/recetas_verificar.py`
- **Aprobación admin + emisión cert** `routers/admin.py`